

Apache Kafka Basics

1. What Is Apache Kafka?

Apache Kafka is a distributed event streaming platform. It stores ordered records in topics and allows producers to publish records and consumers to read them.

Kafka is commonly used for event ingestion, streaming pipelines, application integration, log collection, and real-time analytics.

2. Topics and Partitions

A topic is a logical stream of records. A topic is divided into partitions. Records within a partition have an order, while ordering across different partitions is not generally guaranteed.

Partitions provide parallelism and scale. More partitions can allow more consumers in a consumer group to process data concurrently.

3. Producers

Producers publish records to Kafka topics. A producer can choose a partition explicitly or rely on a partitioning strategy based on the record key.

Using a stable key can ensure related records are sent to the same partition, which is useful when per-key ordering matters.

4. Consumers and Consumer Groups

A consumer reads records from partitions. Consumers can form a consumer group. Within a group, each partition is assigned to at most one active consumer at a time.

If there are more consumers than partitions, some consumers will be idle.

5. Offsets

Each record in a partition has an offset. Consumers use offsets to track their position. Offset commits allow consumers to resume from a known position after restart.

Offset management affects delivery semantics and recovery behavior.

6. Replication

Kafka replicates partitions across brokers for fault tolerance. One replica acts as leader for writes and reads under normal operation, while follower replicas replicate the partition data.

Replication factor and in-sync replica settings affect durability and availability.

7. Delivery Semantics

At-most-once processing may lose records but avoids duplicates under certain designs. At-least-once processing can retry records and may produce duplicates. Exactly-once semantics require stronger coordination and are workload dependent.

Idempotent consumers are often used to make at-least-once delivery safer.

8. Producer Example

```
from kafka import KafkaProducer
import json

producer = KafkaProducer(
    bootstrap_servers=["localhost:9092"],
    value_serializer=lambda v: json.dumps(v).encode("utf-8")
)

producer.send(
    "customer-events",
    key=b"customer-101",
    value={"customer_id": 101, "event": "purchase"}
)

producer.flush()
```

9. Consumer Example

```
from kafka import KafkaConsumer
import json

consumer = KafkaConsumer(
    "customer-events",
    bootstrap_servers=["localhost:9092"],
    group_id="analytics-service",
    auto_offset_reset="earliest",
    value_deserializer=lambda v: json.loads(v.decode("utf-8"))
)

for message in consumer:
    print(message.partition, message.offset, message.value)
```

10. Key Terms

Topic, partition, broker, producer, consumer, consumer group, offset, replication factor, leader, follower, ISR, retention, key, ordering, lag.